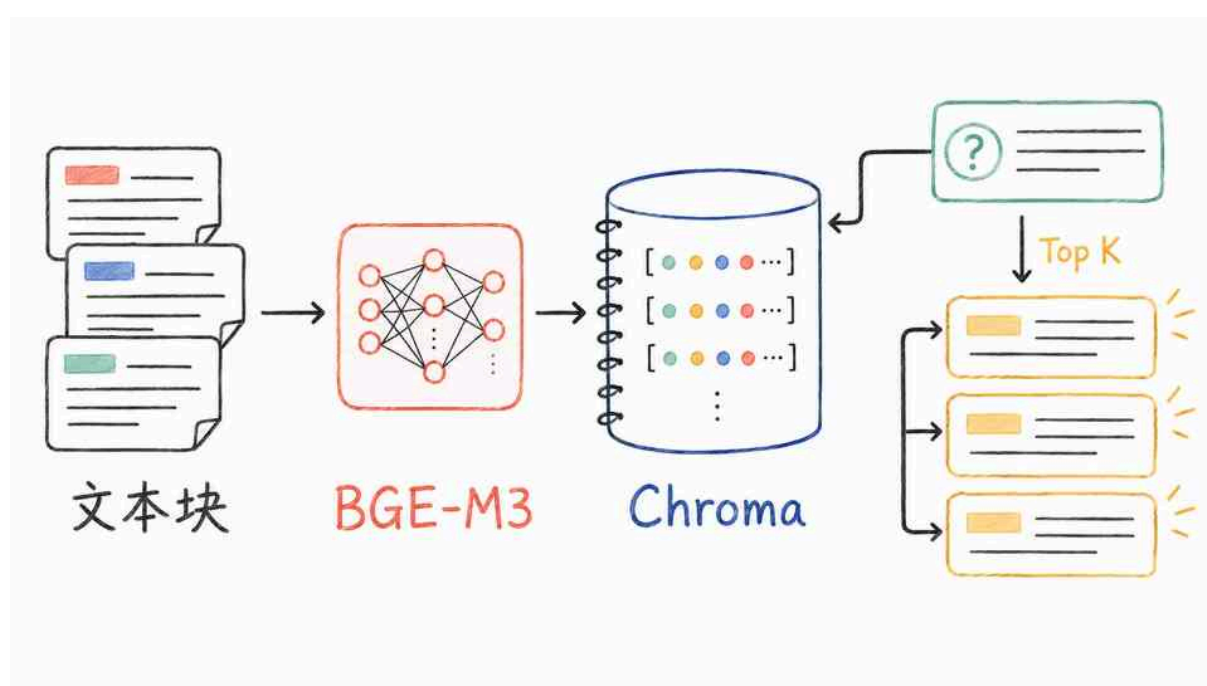


RAG 实战教程（二）：向量相似度、向量数据库解析与 Chroma 实战

Hello, 大家好，我是Ivan.

在上一篇教程中，我们已经把 RAG 的分片、索引、召回、重排和生成都讲了一遍。其中索引比较简单，你只需调用在线 Embedding 接口生成向量，再保存成 JSON 文件，查询时逐条计算相似度。但随之而来的问题是，传入的文件少的时候还可以用，一旦多了，索引的保存和查询都会变得麻烦起来。



这次，我们把索引部分换成一套本地方案。用Ollama 运行 BGE-M3，Chroma 保存文本、向量和元数据。程序重启以后可以直接读取原来的索引，不需要重新处理全部文档。

这次只处理索引和召回，不再重复分片、重排和生成。最后会得到一段可以直接运行的 Python 代码：把文本块写入 Chroma，再根据问题取回 Top K 结果。

一、这次要替换哪一部分

上一次，我们的索引保存在 `index.json` 中，查询时用 NumPy 逐条计算余弦相似度。这种写法方便理解 RAG 的流程，但还不算真正的向量库。

这次保留原来的处理顺序，只替换中间两项：

上一篇：在线 Embedding → JSON 索引 → NumPy 计算相似度
这一篇：本地 BGE-M3 → Chroma → HNSW 近邻检索

这个例子里，我们先不接 LangChain，直接用 Ollama 和 Chroma。代码虽然多一点，但每一步都能清晰看到，等到后面换模型或者排查检索结果时，也知道该从哪里看。

二、向量相似度是怎么计算的

文本经过 Embedding 模型处理以后，会变成一组数字。查询时，用户问题也会经过同一个模型，然后拿查询向量和知识库里的向量计算距离。

为了方便理解，这里先不用 BGE-M3 生成的高维向量，直接看两个三维向量：

```
import numpy as np

vector_a = np.array([3.0, 4.0, 5.0])
vector_b = np.array([6.0, 8.0, 10.0])
```

`vector_b` 正好是 `vector_a` 的两倍。它们的长度不同，但是方向完全一致。如果这两个向量表示两段文本，可以认为它们表达的语义方向很接近。

1. 余弦相似度

余弦相似度比较的是两个向量的方向，公式是：

$$\text{cosine_similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

用 NumPy 计算：

```
def cosine_similarity(a: np.ndarray, b: np.ndarray) -> float:
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

```
print(cosine_similarity(vector_a, vector_b))
```

这段代码的结果接近 1，说明两个向量方向一致。余弦相似度越大，一般表示语义越接近。

向量数据库返回的经常是余弦距离，计算方式为：

$$\text{cosine_distance} = 1 - \text{cosine_similarity}$$

因此在 Chroma 的查询结果里，距离越小越相关。这个数值是距离，不是概率，不能把 0.2 解释成有 80% 的相关性。

2. 欧氏距离

欧氏距离就是两个点在空间中的直线距离：

$$L2(A, B) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

```
def euclidean_distance(a: np.ndarray, b: np.ndarray) -> float:  
    at:  
    return float(np.linalg.norm(a - b))
```

欧氏距离会受到向量长度影响。两个向量方向相同，只要长度不同，距离仍然可能比较大。余弦相似度更关心方向，所以文本语义检索里经常使用余弦距离。

3. 点积

点积是对应位置相乘后相加：

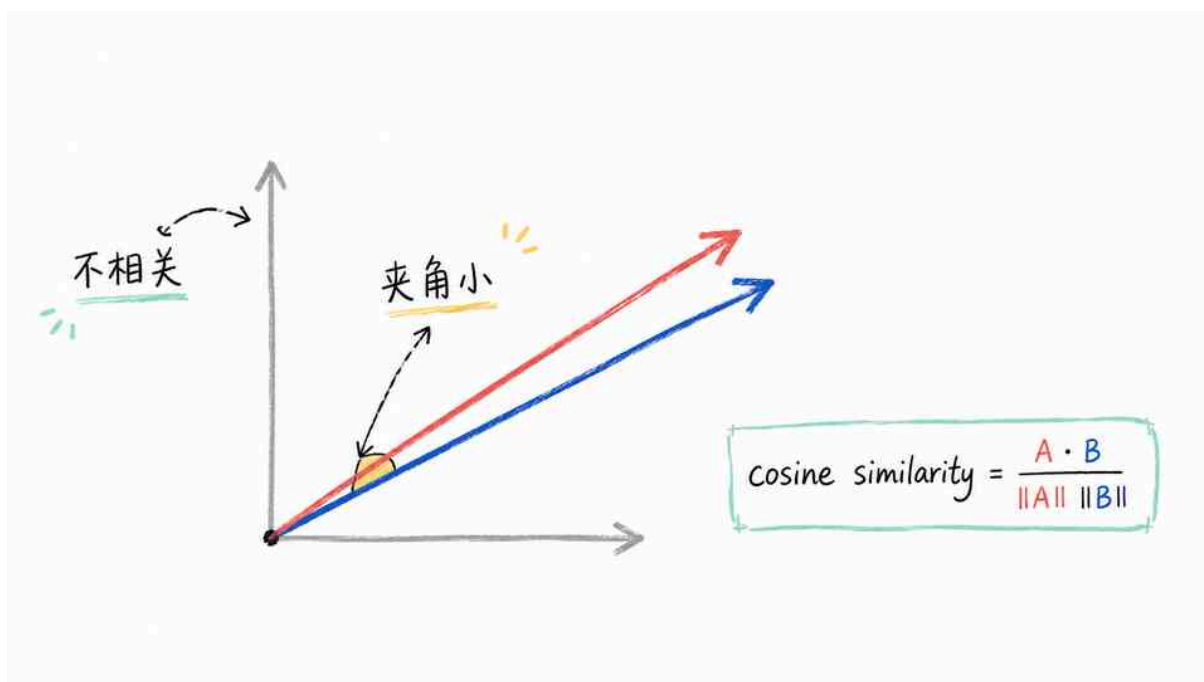
```
def dot_product(a: np.ndarray, b: np.ndarray) -> float:  
    return float(np.dot(a, b))
```

向量归一化以后，点积和余弦相似度的排序结果相同。没有归一化时，点积还会受到向量长度影响。

三种计算方式可以简单看成下面这样：

计算方式	主要比较什么	结果怎样看
余弦相似度	向量方向	越大越相似
余弦距离	向量方向	越小越相似
欧氏距离	空间中的直线距离	越小越相似
点积	方向和长度	一般越大越相似

实际项目里不需要自己遍历所有向量计算这些值，Chroma 会按照集合的距离配置完成检索。不过理解这里的区别以后，再看 `distances` 就不会把大小关系弄反。



三、向量数据库负责什么

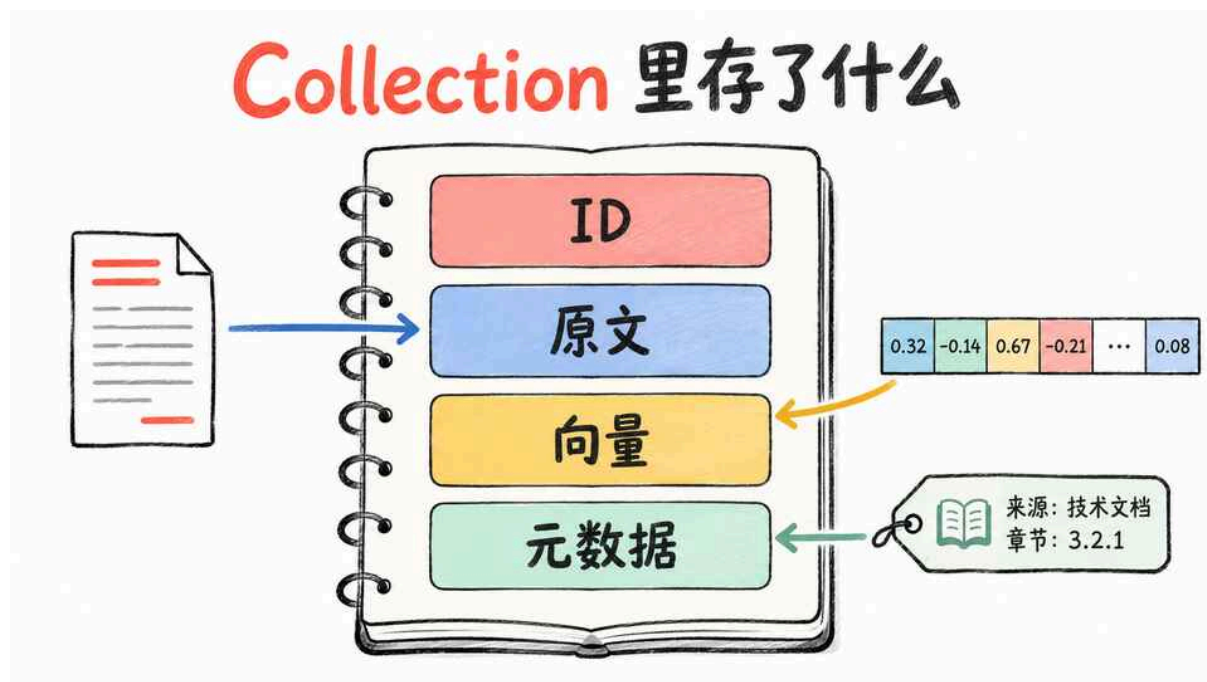
向量可以保存在 JSON、NumPy 文件或者普通数据库里，但是保存下来以后，还要解决查询速度。假设知识库里有几十万个 Chunk，每次提问都逐条计算距离，数据越多，等待时间就越长。

向量数据库会给向量建立索引，并提供新增、查询、更新和删除等操作。一次完整的 Chunk 记录通常包含下面几项：

```
{
  "id": "manual-001-chunk-03",
  "document": "空压机累计运行 500 小时后，需要检查润滑油状态。",
  "embedding": [0.0127, -0.0314, 0.0089],
  "metadata": {
    "source": "空压机维护手册",
  }
}
```

```
"section": "润滑系统"
}
```

向量用于计算距离，`document` 是检索后真正要交给大模型的原文，`metadata` 用来记录文件名、章节、页码和权限等信息。只留下向量的话，数据库虽然能找到最近的点，却拿不出对应的资料内容。



1. Collection 可以理解成一组向量数据

Chroma 使用 Collection 管理数据。一个 Collection 里放一批使用相同 Embedding 模型和相同维度生成的向量，也会保存对应的 ID、原文和元数据。

常用操作有这些：

- `add`：添加数据，ID 已存在时会报错或忽略，具体行为取决于版本。
- `upsert`：ID 不存在就新增，已经存在就更新。
- `query`：传入查询向量，计算距离并返回 Top K。
- `get`：按照 ID 或过滤条件读取数据，不计算向量距离。
- `delete`：删除指定数据。

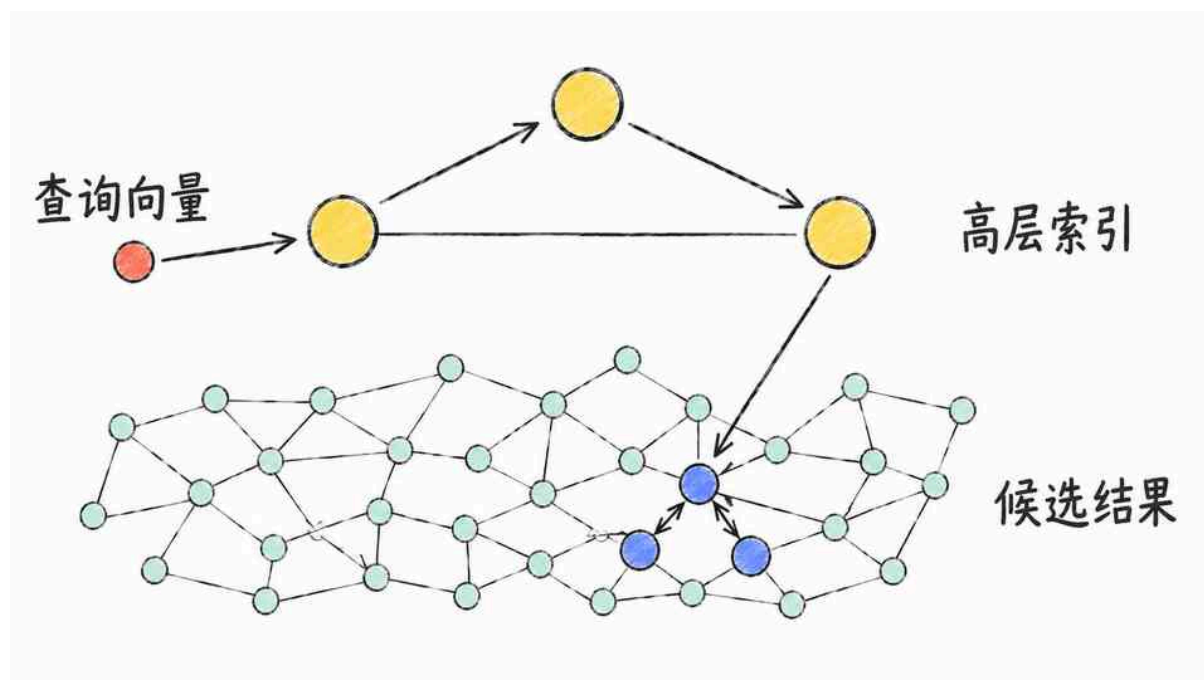
后面的示例使用 `upsert`，重复运行时不会留下多份相同的 Chunk。

2. HNSW 用来加快近邻检索

如果数据库每次都拿查询向量和全部数据逐条比较，做的是暴力检索。结果准确，但是数据量大以后会比较慢。

Chroma 的单机 Collection 使用 HNSW 建立近邻索引。可以把它理解成一张分层的图：查询先在较高的层级快速找到大致区域，再进入更细的层级寻找附近的向量。它通常不需要扫描全部数据，查询速度会快很多，不过结果属于近似最近邻。

HNSW 的参数会影响索引体积、构建时间、查询速度和召回率。刚开始做 Demo 时使用默认参数即可，这一篇只指定距离方式为 `cosine`。



四、准备运行环境

这次使用 Python、Ollama 和 Chroma。Ollama 在本地运行 BGE-M3，文档和问题不需要发给在线 Embedding 接口。

1. 安装 Python 依赖

建议使用 Python 3.10 及以上版本，新建一个虚拟环境后安装依赖：

```
python -m venv .venv
```

Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

macOS 或 Linux:

```
source .venv/bin/activate
```

安装 Chroma 和 Ollama 的 Python SDK:

```
python -m pip install chromadb ollama
```

2. 下载 BGE-M3

安装并启动 Ollama 后, 执行:

```
ollama pull bge-m3
```

BGE-M3 是一个支持多语言和长文本的 Embedding 模型, 中文文档也可以直接使用。模型下载完成后, 可以用下面的命令确认:

```
ollama list
```

Ollama 默认在本机的 11434 端口提供服务。这里不需要运行 `ollama run bge-m3` 并保持对话窗口, Python SDK 调用 Embedding 接口时会直接加载模型。

五、用 BGE-M3 生成向量

先写一个最小示例, 确认本地模型可以正常调用:

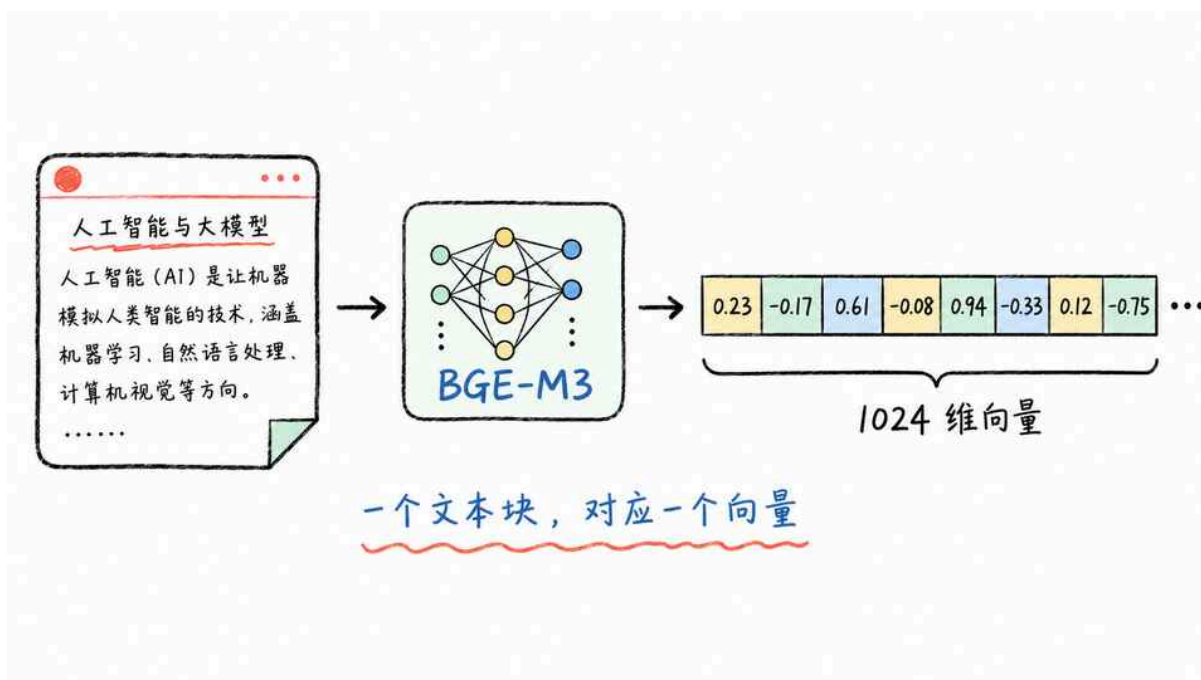
```
import ollama

response = ollama.embed(
    model="bge-m3",
    input="空压机需要定期检查润滑油。",
)

vector = response["embeddings"][0]

print("向量数量: ", len(response["embeddings"]))
print("向量维度: ", len(vector))
print("前 5 个值: ", vector[:5])
```

`embeddings` 是一个二维列表。即使只传入一段文本，返回结果也保留了批量结构，所以需要通过 `[0]` 取出第一条向量。



如果已经有多个文本块，可以直接批量传入：

```
import ollama

texts = [
    "空压机累计运行 500 小时后，需要检查润滑油状态。",
    "冷却水温度超过 35°C 时，应检查循环泵和散热器。",
    "传送带出现跑偏时，应先停机并检查两侧张紧机构。",
]

response = ollama.embed(
    model="bge-m3",
    input=texts,
)

embeddings = response["embeddings"]

print("文本块数量：", len(texts))
print("向量数量：", len(embeddings))
print("每条向量的维度：", len(embeddings[0]))
```

文本很多时不要一次全部传入。可以按固定批次处理，避免单次请求过大：

```
import ollama

def embed_texts(texts: list[str], batch_size: int = 16) -> list[list[float]]:
    all_embeddings = []

    for start in range(0, len(texts), batch_size):
        batch = texts[start:start + batch_size]
        response = ollama.embed(
            model="bge-m3",
            input=batch,
        )
        all_embeddings.extend(response["embeddings"])

    return all_embeddings
```

这里的 `batch_size` 只是一次传多少个文本块，不会改变单条向量的维度。批次大小需要结合机器内存、文本长度和模型接口限制来调整。

入库和查询都要使用同一个 Embedding 模型。这里写入 Chroma 的文本块由 BGE-M3 处理，用户问题也必须继续使用 BGE-M3。中途换了模型，即使向量维度相同，原来的索引也应该重新生成。

六、把向量和原文存进 Chroma

有了向量以后，接下来把它们存进向量数据库。这次使用 Chroma 的本地持久化模式，数据会保存到项目中的 `chroma_data` 目录，关闭程序后仍然可以继续查询。

```
import chromadb

client = chromadb.PersistentClient(path="./chroma_data")

collection = client.get_or_create_collection(
    name="equipment_manual",
    embedding_function=None,
    configuration={
```

```

        "hsw": {
            "space": "cosine",
        },
    },
)

```

这里把 `embedding_function` 设为 `None`，因为向量已经由 Ollama 生成，后面会手动传给 Chroma。`space` 使用 `cosine`，表示按余弦距离检索。Chroma 的单机集合默认使用 HNSW 索引，适合做高维向量的近邻搜索。

准备几条已经分好的数据：

```

documents = [
    "空压机累计运行 500 小时后，需要检查润滑油状态；发现颜色发黑或杂质明显时应提前更换。",
    "冷却水温度超过 35℃ 时，应检查循环泵、散热器和管路是否堵塞。",
    "传送带出现跑偏时，应先停机，再检查两侧张紧机构和滚筒位置。",
    "设备每次维护完成后，需要在系统中填写维护时间、处理内容和操作人员。",
    "高温区域作业前应确认防护用品齐全，并由现场负责人完成安全检查。",
]

metadatas = [
    {"source": "空压机维护手册", "section": "润滑系统"},
    {"source": "冷却系统手册", "section": "温度异常"},
    {"source": "传送设备手册", "section": "跑偏处理"},
    {"source": "设备管理制度", "section": "维护记录"},
    {"source": "安全作业规范", "section": "高温作业"},
]

ids = [f"chunk-{index}" for index in range(len(documents))]

```

生成向量并写入集合：

```

document_embeddings = embed_texts(documents)

collection.upsert(
    ids=ids,
    documents=documents,
)

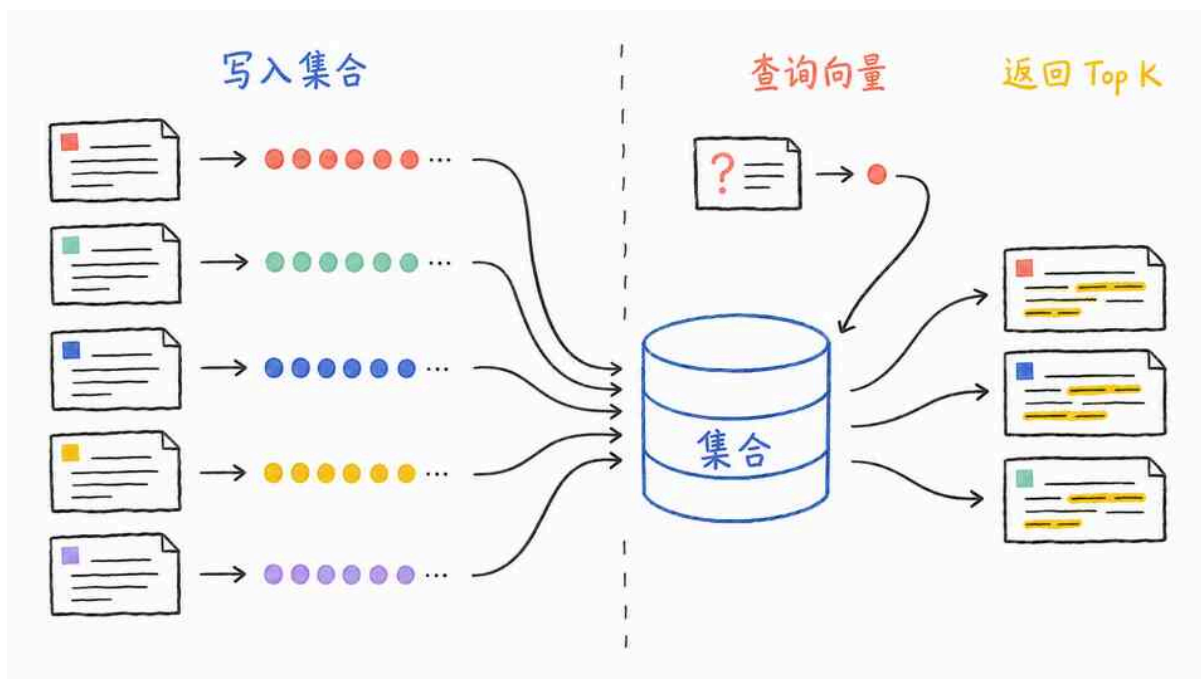
```

```

embeddings=document_embeddings,
metadatas=metadatas,
)

print("当前文本块数量：", collection.count())

```



这里保存的不只是向量，还包括原始文本和元数据。

向量只参与距离计算，检索完成以后，原文会被交给大模型。元数据可以记录来源文件、章节、页码和更新时间，后面生成引用或限定检索范围时都会用到。

`upsert` 会根据 ID 新增或更新数据，重复运行示例不会因为 ID 已存在而留下多份相同文本。在真实项目中，ID 最好由文件路径、页码、块序号或内容哈希稳定生成，不要每次随机创建。

七、根据用户问题检索 Top K 文本

知识已经入库，现在处理用户问题：

```
query = "空压机的油多久检查一次？"
```

这个问题也要使用 BGE-M3 转成向量：

```
query_embedding = embed_texts([query])[0]
```

然后交给 Chroma 查询：

```
results = collection.query(
    query_embeddings=[query_embedding],
    n_results=3,
    include=["documents", "metadatas", "distances"],
)
```

`n_results=3` 表示返回距离最近的三个文本块，也就是常说的 Top K。打印结果：

```
for document, metadata, distance in zip(
    results["documents"][0],
    results["metadatas"][0],
    results["distances"][0],
):
    print(f"距离: {distance:.4f}")
    print(f"来源: {metadata['source']} / {metadata['section']}")
    print(f"内容: {document}")
    print("-" * 50)
```

正常情况下，与空压机润滑油相关的文本块会排在前面。具体距离会受到模型、Chroma 版本和数据内容影响，所以这里不写固定结果。

距离越小，通常表示越相关

我们在集合中选择的是余弦距离。余弦相似度越高，两条文本的方向越接近；Chroma 返回的是距离，可以简单理解为数值越小越接近。

不过这个距离不是概率，不能把 `0.2` 理解成“有 80% 相关”。项目里如果要设置拒答阈值，需要准备一批真实问题和标准答案，再根据评测结果决定，不能直接照抄别人的数值。

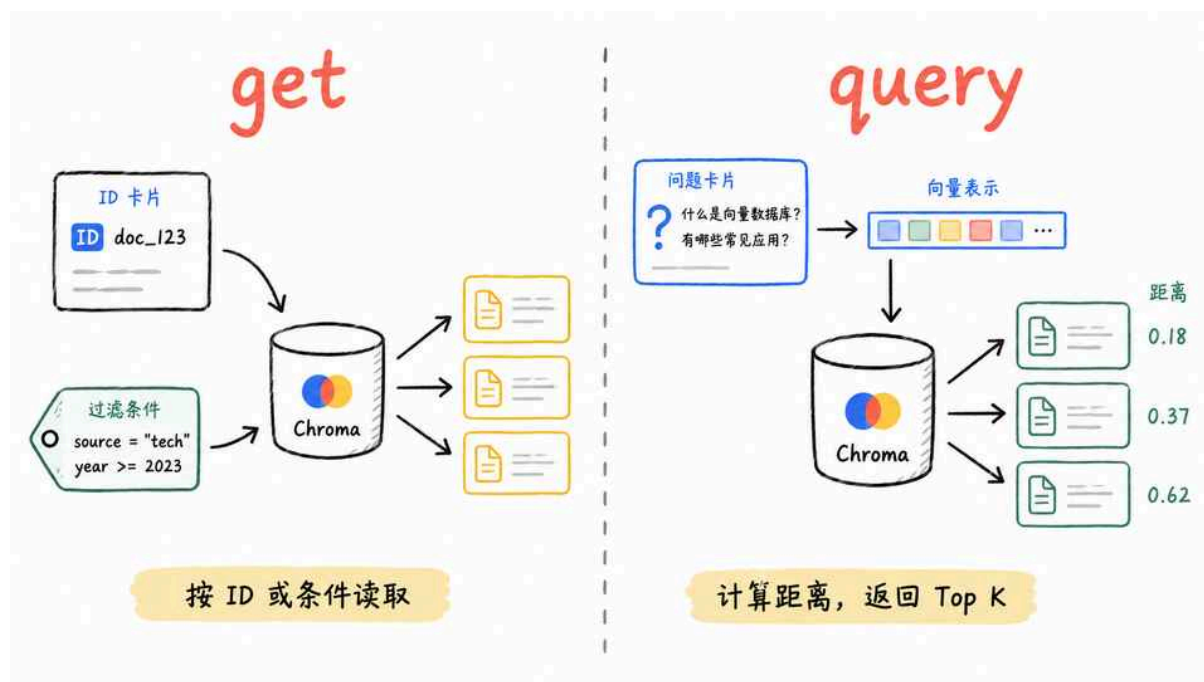
get 和 query 不一样

调试 Chroma 时还会看到 `get`：

```
stored = collection.get(
    limit=5,
```

```
include=["documents", "metadatas"],
)
```

`get` 只是按 ID 或过滤条件读取已经保存的数据，不会计算语义相似度。`query` 才会使用查询向量做近邻检索并返回距离。



八、完整代码

把前面的内容合在一起，新建 `rag_vector_search.py`：

```
import chromadb
import ollama

EMBEDDING_MODEL = "bge-m3"
DATABASE_PATH = "./chroma_data"
COLLECTION_NAME = "equipment_manual"

def embed_texts(
    texts: list[str],
    batch_size: int = 16,
) -> list[list[float]]:
    """批量生成文本向量。"""
```

```

all_embeddings = []

for start in range(0, len(texts), batch_size):
    batch = texts[start:start + batch_size]
    response = ollama.embed(
        model=EMBEDDING_MODEL,
        input=batch,
    )
    all_embeddings.extend(response["embeddings"])

return all_embeddings

def create_collection():
    """创建或读取本地 Chroma 集合。"""
    client = chromadb.PersistentClient(path=DATABASE_PATH)

    return client.get_or_create_collection(
        name=COLLECTION_NAME,
        embedding_function=None,
        configuration={
            "hnsw": {
                "space": "cosine",
            },
        },
    )

def build_knowledge_base(collection) -> None:
    """生成向量并写入知识库。"""
    documents = [
        "空压机累计运行 500 小时后，需要检查润滑油状态；发现颜色发黑或杂质明显时应提前更换。",
        "冷却水温度超过 35°C 时，应检查循环泵、散热器和管路是否堵塞。",
        "传送带出现跑偏时，应先停机，再检查两侧张紧机构和滚筒位置。",
        "设备每次维护完成后，需要在系统中填写维护时间、处理内容和操作人员。",
    ]

```

```

        "高温区域作业前应确认防护用品齐全，并由现场负责人完成安全检
查。",
    ]

    metadatas = [
        {"source": "空压机维护手册", "section": "润滑系统"},
        {"source": "冷却系统手册", "section": "温度异常"},
        {"source": "传送设备手册", "section": "跑偏处理"},
        {"source": "设备管理制度", "section": "维护记录"},
        {"source": "安全作业规范", "section": "高温作业"},
    ]

    ids = [f"chunk-{index}" for index in range(len(document
s))]
    embeddings = embed_texts(documents)

    collection.upsert(
        ids=ids,
        documents=documents,
        embeddings=embeddings,
        metadatas=metadatas,
    )

def search(collection, query: str, top_k: int = 3) -> list
[dict]:
    """根据用户问题检索相关文本块。"""
    query_embedding = embed_texts([query])[0]

    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=top_k,
        include=["documents", "metadatas", "distances"],
    )

    items = []

    for document, metadata, distance in zip(

```

```

        results["documents"][0],
        results["metadatas"][0],
        results["distances"][0],
    ):
        items.append(
            {
                "document": document,
                "metadata": metadata,
                "distance": distance,
            }
        )

    return items

if __name__ == "__main__":
    collection = create_collection()

    if collection.count() == 0:
        build_knowledge_base(collection)

    user_query = "空压机的油多久检查一次？"
    search_results = search(collection, user_query, top_k=
3)

    print(f"问题: {user_query}")
    print(f"知识库文本块数量: {collection.count()}")
    print()

    for index, item in enumerate(search_results, start=1):
        metadata = item["metadata"]
        print(f"结果 {index}")
        print(f"距离: {item['distance']:.4f}")
        print(f"来源: {metadata['source']} / {metadata['section']}")
        print(f"内容: {item['document']}")
        print("-" * 50)

```

运行：

```
python rag_vector_search.py
```

第一次运行时，程序会生成向量并写入 `chroma_data`。再次运行时，集合中已经有数据，会直接进入查询，不再重复调用 BGE-M3 处理同一批文本。

这段代码完成了下面这条链路：

```
文本块
→ 批量生成向量
→ 保存向量、原文和元数据
→ 用户问题向量化
→ Chroma 相似度检索
→ 返回 Top K 原文
```

这次没有调用大模型，程序只打印检索结果。这样更方便检查 BGE-M3 和 Chroma 到底找回了哪些内容。确认召回结果正常后，再接回上一篇的重排和生成代码即可。

九、几个容易出错的地方

1. 知识库和查询换了 Embedding 模型

这会直接破坏向量之间的可比性。项目里应该把模型名称写进配置，并和集合版本绑定。更换模型后，需要重新生成整个集合的向量。

2. 只保存向量，没有保存原文

向量负责检索，原文负责回答。只保存向量以后，即使找到了最近的坐标，也没有内容可以交给大模型。实际项目还应该保存来源文件、章节和页码，方便展示引用。

3. 每次启动程序都重新构建知识库

文档解析、分块和向量化通常属于离线流程。数据没有变化时，没有必要在每次提问前重新执行。使用 `PersistentClient` 后，可以把构建脚本和查询脚本分开，只在文档新增或修改时更新相关文本块。

4. Top K 设置得太大

返回更多文本不一定能让回答更准确。无关内容增加以后，大模型反而更难判断哪些信息应该使用，输入 Token 和调用成本也会增加。Top K 可以先从 3 到 5 开始，再结合召回率和最终回答效果调整。

5. 只看向量维度，不看检索结果

向量成功生成，只能说明接口已经跑通。Embedding 模型是否适合自己的文档，还要通过真实问题验证。尤其是专业术语、简称和内部产品名称，通用模型不一定能处理好，需要补充同义词、调整分块，或者更换适合中文和垂直领域的模型。

十、总结

到这里，我们已经能把文本写入 Chroma 并完成一次向量检索。下一篇就继续往下看，向量数据库是怎么用 KNN、IVF 和 HNSW 在大量向量里找到 Top K 结果的，也会用 Faiss 把这些检索算法跑一遍。